

基于 RK3588 的智能电梯监控兼广告机系统

摘要

本作品面向电梯轿厢运行安全、乘客体验和远程运维需求，设计了一套由单片机采集端、边缘计算主控、本地人机界面、视频 AI 识别、云端 MQTT 平台和网页监控端组成的智能电梯安全监测与云端联动系统。系统以单片机采集的楼层、温度、湿度等数据为基础，采用带帧头、类型、长度、载荷、CRC16 校验和帧尾的数据帧进行传输，边缘主控完成数据解析、去重、状态更新和本地显示。当前工程中使用 ElevatorEnvSimulator 产生与单片机协议一致的数据流，便于在无硬件联调条件下验证完整链路；接入实物时只需保持帧格式不变即可替换为真实串口数据。

边缘端采用 C++17 模块化架构，集成 LVGL 图形界面、GStreamer 音视频管线、OpenCV 图像处理和 RKNN NPU 推理能力。系统可在本地屏幕显示楼层、温湿度和报警状态，同时保留广告播放区域；摄像头链路以 1920×1080、30 fps 采集画面，经 RKNN 模型推理后叠加目标框并推送到 RTSP 服务。云端部分通过 Python 桥接程序将 C++ 本地 Unix Datagram Socket 与 EMQX MQTT 主题连接起来，实现设备在线状态、环境数据、媒体状态上报，以及报警、广告切换、摄像头启停等远程控制。网页监控端支持 MQTT over WebSocket，展示实时数据和视频画面，并可上传广告视频后下发到边缘端自动播放。

作品的核心价值在于把传统电梯状态显示扩展为“数据采集、边缘智能、云端运维、媒体联动”的闭环系统。相比单一的楼层显示或传感器报警方案，本系统同时关注安全告警、实时可视化、远程运维和内容投放，具备较强的工程可扩展性。

第一部分 作品概述

1.1 功能与特性

① **环境与运行数据采集：**采集当前楼层、轿厢温度、湿度、帧序号和设备时间戳，使用 CRC16 校验提高通信可靠性。

② **本地交互显示：**基于 LVGL 构建电梯显示界面，实时刷新楼层、温湿度和警报状态，并提供广告显示区域。

③ **云端远程监控：**通过 MQTT 上报 telemetry、state、media 三类主题，网页端实时订阅并展示设备状态。

④ **远程控制闭环：**网页端可下发报警触发/解除、广告切换/停止、摄像头启动/停止等指令，边缘端执行后回传状态。

⑤ **视频 AI 能力：**摄像头画面经过 RKNN 模型推理和 OpenCV 目标框叠加后推流，可扩展乘客检测、异常行为识别和危险物品识别。

⑥ **广告动态投放：**云端上传 MP4 文件，边缘端下载后通过 GStreamer 循环播放，并在停止时自动清屏恢复检测能力。

1.2 应用领域

本作品适用于住宅楼宇、商场、医院、学校、办公园区和工业园区等多种电梯场景。对于物业运维人员，系统可提供设备在线状态、运行环境和媒体状态的远程可视化，减少现场巡检压力；对于公共建筑，系统可作为安全监管、应急告警和信息发布终端；对于老旧电梯改造，系统可在不大幅改变既有控制柜的情况下，通过外挂采集板和边缘计算盒子叠加智能监控能力。



1.3 主要技术特点

① **分层解耦架构：**采集、解析、UI、媒体、云端桥接和网页前端相互独立，模块边界清晰。

② **可靠数据协议：**采用固定帧头帧尾、长度字段和 CRC16 校验，解析器使用状态机逐字节处理，适合串口类不稳定数据流。

③ **边缘 AI 与媒体协同：**广告播放时暂停检测，广告停止或异常结束后恢复检测，降低资源冲突。

④ **本地 IPC + MQTT 双层通信：**边缘 C++ 与 Python 使用 Unix Datagram Socket，Python 与云端使用 MQTT，兼顾实时性和网络扩展性。

⑤ **跨平台构建：**CMake 支持 x86 主机调试和 ARM64 交叉编译，RKNN 在目标板启用，在非目标架构自动降级。

1.4 主要性能指标

指标项	设计/测试值	说明
采集帧周期	2.5 s	当前模拟单片机周期，可按真实传感器采样率调整

串口协议	AA/type/len/payload/CRC 16/55	支持长度保护和 CRC 错误 丢弃
数据范围	楼层 1-10；温度 20.0- 35.8° C；湿度 40-80%	用于联调与展示的单片机 采集样例范围
视频采集	1920×1080 @ 30 fps	GStreamer v4l2src 摄像 头链路
视频推流	H.264 RTSP，约 3-5 Mbps	面向云端 WebRTC/RTSP 服 务转发
AI 推理参数	置信度 0.35；NMS 0.45； 最多 64 框	RKNN 检测模型后处理参数
云端消息	MQTT QoS 1	telemetry/state/media/c ommand 主题
命令响应	本地 IPC 100 ms 轮询超 时	报警和媒体命令可在本地 快速触发

1.5 主要创新点

- ① 将电梯数据采集、AI 视频检测、广告投放和云端运维集成到同一边缘系统中，形成多功能闭环。
- ② 媒体状态通过 MQTT 回传，网页端不仅能下发命令，还能感知广告、摄像头等执行结果。
- ③ 摄像头检测与广告投放互斥调度，兼顾运算资源、显示效果和系统稳定性。

1.6 设计流程

设计流程为：需求分析 -> 系统分层架构设计 -> 单片机通信协议设计 -> C++ 边缘端模块开发 -> LVGL 本地界面设计 -> MQTT 云端桥接 -> 视频 AI 与广告投放集成 -> ARM64 交叉编译部署 -> 功能与性能联调。

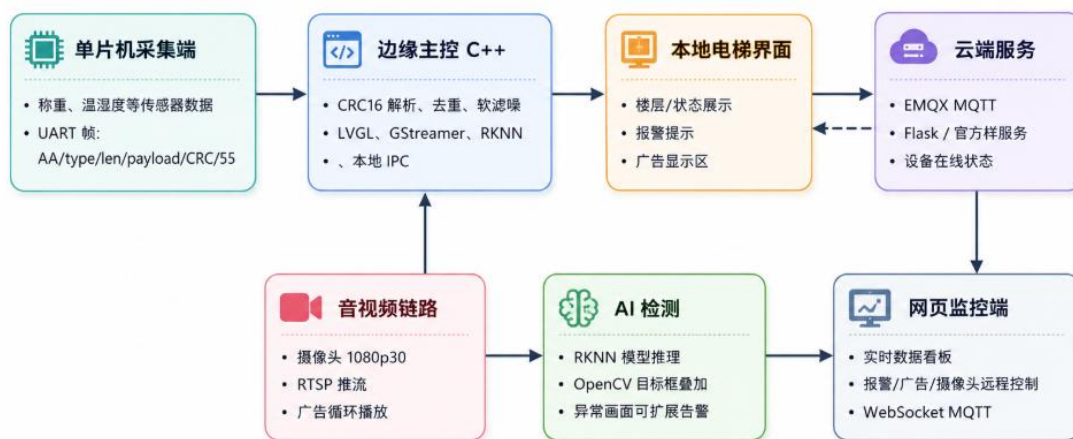


第二部分 系统组成及功能说明

2.1 整体介绍

给出系统整体框图，各子模块标注清楚，并进行整体的文字说明，需要表达出各模块之间的关系。

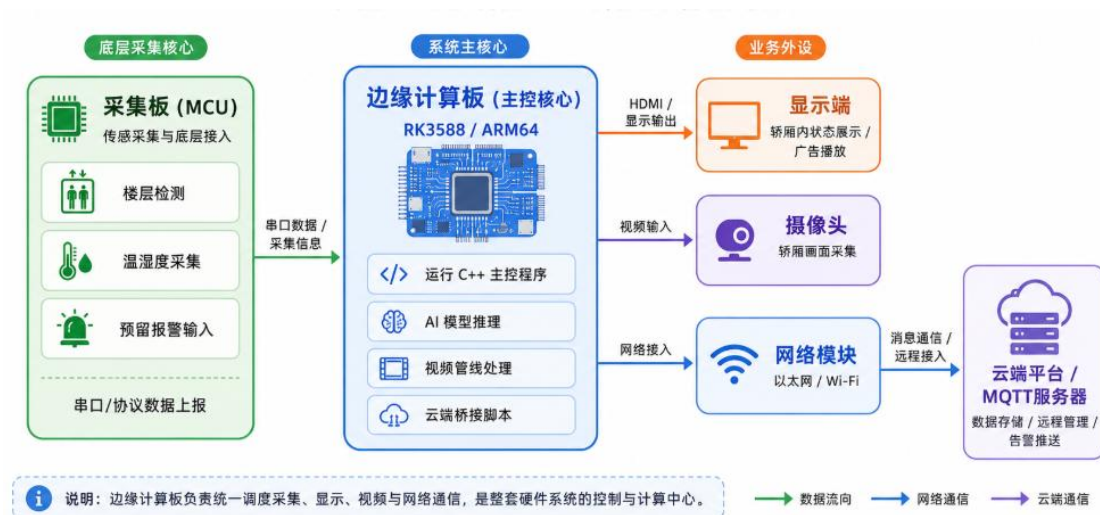
系统由单片机采集端、边缘主控、媒体与 AI 模块、云端服务和网页监控端组成。单片机负责采集电梯运行和轿厢环境信息，边缘主控解析数据并驱动本地显示；同时通过本地 Socket 把数据交给 Python 桥接程序，桥接程序再发布到 MQTT 云端。网页端既订阅实时状态，也通过 command 主题下发控制命令。摄像头画面由边缘端完成采集、AI 识别和推流，广告文件则由云端上传后下发到边缘端播放。



2.2 硬件系统介绍

2.2.1 硬件整体介绍：

硬件系统采用“采集板 + 边缘计算板 + 显示/摄像头/网络外设”的结构。采集板以单片机为核心，连接楼层检测、温湿度采集和预留报警输入；边缘计算板面向 RK3588/ARM64 环境，负责运行 C++ 主控程序、AI 模型、视频管线和云端桥接脚本；显示端用于轿厢内状态展示和广告播放；摄像头用于轿厢画面采集；网络模块通过以太网或 Wi-Fi 接入局域网和 MQTT 服务器。



2.3 软件系统介绍

2.3.1 软件整体介绍（含 PC 端或云端，结合关键图片）；

软件分为边缘端 C++、云端桥接 Python、网页监控端三部分。C++ 程序启动后初始化 UI、媒体服务和本地命令监听；单片机数据经解析器处理后同步送往 LVGL 界面和 telemetry Socket；Python 桥接程序将 telemetry 转为 MQTT 消息，同时接收网页端 command 指令并转为本地控制命令；网页端订阅 telemetry、state、media 主题实现可视化。



2.3.2 软件各模块介绍

本系统的软件不是把所有功能写在一个主循环里，而是按“数据入口、业务调度、界面显示、媒体处理、云端桥接、网页交互”进行分层。单片机数据从串口协议进入 C++ 边缘端，经过解析器转换成结构体数据后，一路送往 LVGL 本地界面，另一路通过本地 Socket 交给 Python 程序上云；云端控制命令则沿相反方向从网页进入 MQTT，再由 Python 转成本地命令交给 C++ 执行。这样的结构使传感器采集、屏幕显示、视频处理和云端控制互不耦合，后期替换真实硬件或扩展新功能时不会牵动全局。

主控调度模块：主控模块由 `ApplicationRuntime` 类承担，是整个边缘端程序的“总控制台”。程序启动后，`main` 只负责创建 `ApplicationRuntime`、注册 SIGINT 信号并进入等待循环；具体的初始化顺序、停止顺序和模块之间的回调连接都在 `ApplicationRuntime::start` 与 `stop` 中完成。该模块启动 LVGL 界面、注册环境数据回调、启动本地命令监听、启动摄像头服务，并创建单片机数据源。停止时则先关闭数据生产线程，再关闭命令监听、媒体服务和 UI 线程，避免程序退出时出现线程残留或资源未释放。

协议解析模块：协议解析模块由 `ElevatorFrameParser` 完成，核心函数是 `parseByte`。该函数按字节接收单片机数据流，通过 `WAIT_HEADER`、`WAIT_TYPE`、`WAIT_LEN`、`WAIT_DATA`、`WAIT_CRC_LOW`、`WAIT_CRC_HIGH`、`WAIT_TAIL` 七个状态逐步判断一帧数据是否完整。帧格式使用 `0xAA` 作为帧头，`0x55` 作为帧尾，中间包含类型、长度、载荷和 CRC16 校验。只有 CRC 校验正确且帧尾匹配时，解析器才会调用 `processValidFrame`，把 payload 映射为楼层、温度、湿度、帧序号和设备时间戳。这样可以过滤串口干扰、错位数据和残缺帧。

单片机数据源模块：当前工程中的 `ElevatorEnvSimulator` 用来模拟单片机采集端，它按真实协议构造数据帧，并以 8 ms 的字节间隔逐字节喂给解析器，模拟 UART 接收过程。模拟数据包括楼层 1 至 10 循环、温度小数递增、湿度周期变化、`frameSeq` 递增和 `deviceTimestampMs` 增长。该模块的作用不是替代硬件，而是在硬件未完全接入时验证上层链路。后续接入真实单片机时，只需要把模拟器替换为串口读取任务，仍然调用 `parser.parseByte` 即可。

本地界面模块：本地 UI 由 `ElevatorUiController` 和 `SquareLine` 生成的 LVGL 页面组成。`ui_Screen1.c` 中定义了广告显示区、信息显示区、楼层 Label、温湿度 Label 和报警 Label。`ElevatorUiController` 负责在独立 GUI 线程中初始化 LVGL 和 SDL/KMS 显示环境，并通过 `postEnvData`、`triggerAlert`、`cancelAlert` 接收其他线程传来的业务数据。由于 LVGL 控件不适合被多个线程直接操作，模块内部使用队列缓存数据和报警命令，再由 GUI 线程在 `applyPendingData` 中统一刷新控件，从而保证界面更新的线程安全。

本地通信模块: C++ 边缘端和 Python 云端桥接端之间通过 Unix Datagram Socket 通信。UdpTelemetryClient 负责把环境数据和媒体状态组装成 JSON，发送到 /tmp/elevator_telemetry.sock；UdpCommandServer 绑定 /tmp/elevator_cmd.sock，循环接收 Python 发来的本地控制命令。命令采用简单字符串协议，例如 ALARM_TRIGGER 触发报警，PLAY_AD:path 播放广告，START_CAMERA 启动摄像头。这样设计的好处是 C++ 主控无需直接依赖 MQTT SDK，云端协议变化也主要由 Python 桥接层处理。

媒体管理模块: MediaManager 是摄像头服务和广告播放服务的协调者。它对上提供 startCamera、stopCamera、playAdLoop、stopAd 等接口，对下分别调用 CameraStreamService 和 VideoAdPlayer。广告播放和摄像头 AI 检测会共享显示与计算资源，因此 MediaManager 在开始播放广告前会关闭 detectionEnabled，广告停止或失败后再恢复检测。媒体模块产生的 Starting、Playing、Stopped、Restarting、Failed 等状态会通过回调上报给 ApplicationRuntime，再经 UdpTelemetryClient 上传到云端，使网页端能看到命令执行结果。

摄像头推流模块: CameraStreamService 使用 GStreamer 建立两段管线。第一段管线从 /dev/video21 读取 1920×1080、30 fps 的 MJPEG 摄像头画面，经 jpegdec 和 videoconvert 转成 BGR 帧并送入 appsink；第二段管线从 appsrc 接收处理后的 BGR 帧，经硬件 H.264 编码后推送到 rtsp://192.168.100.10:8554/live。C++ 在线程中不断拉取 sample，转换为 OpenCV Mat，必要时调用 AI 模型推理，再把叠加结果推回 appsrc。模块还会监控 GStreamer bus，一旦出现 ERROR 或 EOS，会上报失败并尝试重启管线。

AI 推理模块: RknnDetector 封装 RKNN 模型加载、输入预处理、推理执行和输出后处理。模型路径为 /home/elf/elevator_app/best_quantized.rknn，初始化时读取模型文件、创建 rknn_context、查询输入输出张量信息并设置 NPU 核心。推理前，模块用 letterbox 方法把摄像头 BGR 图像缩放到模型输入尺寸，保持比例并记录缩放参数；推理后，模块兼容常见 YOLO 单输出布局，计算置信度、类别和边界框，再通过 NMS 去除重复框。CameraStreamService 收到检测框后用 OpenCV 在视频帧上绘制矩形和置信度标签。

广告播放模块: VideoAdPlayer 使用 GStreamer playbin 播放本地 MP4 文件，视频输出通过 kmssink 显示到指定矩形区域，对应轿厢屏幕中的广告区。playLoop 在播放前会检查文件是否存在，并把文件路径转换为 URI；播放过程中监听 GStreamer 状态变化，如果收到 EOS，则 seek 到开头继续循环播放，实现广告循环投放。stop 会把 pipeline 切到 NULL 并回收 bus、pipeline 等资源；clearVideoArea 通过播放一帧黑色视频清空广告区域，避免停止广告后屏幕保留最后一帧。

云端桥接模块：frontend_py/mqtt_client.py 是边缘设备和云端 MQTT 平台之间的桥。程序启动后连接 EMQX，订阅 elevator/001/command，并发布 retained 在线状态。它同时绑定 /tmp/elevator_telemetry.sock，持续接收 C++ 发来的 JSON 数据：普通环境数据发布到 elevator/001/telemetry，媒体状态发布到 elevator/001/media。收到 command 主题消息后，Python 会解析 action 字段，并转换为本地 Socket 命令，例如 sound_alarm 转为 ALARM_TRIGGER，switch_ad 则先下载视频文件再发送 PLAY_AD:path 给 C++。

网页监控模块：云端.html 是面向用户的监控与控制界面，使用 MQTT over WebSocket 连接 EMQX。网页订阅 telemetry、state、media 三类主题，实时刷新楼层、温度、设备在线状态和媒体状态，同时通过 iframe 显示 RTSP/WebRTC 转发后的视频画面。控制区可以下发报警、解除报警、启动/停止摄像头、停止广告等 MQTT 指令。广告投放流程则由网页先把 MP4 上传到 Flask 文件服务，获取下载 URL 后再发布 switch_ad 命令，让设备端自动下载并播放。

从数据流角度看，环境数据链路为“单片机/模拟器 -> ElevatorFrameParser -> ElevatorUiController + UdpTelemetryClient -> mqtt_client.py -> MQTT -> 网页”；控制命令链路为“网页 -> MQTT command -> mqtt_client.py -> UdpCommandServer -> ApplicationRuntime -> UI/MediaManager”。这两条链路一上一下形成闭环，使系统既能展示状态，也能执行远程控制。下表对各模块的关键函数和输入输出进行汇总，便于和代码文件对应。

软件模块	关键函数/类	输入	输出/作用
主控调度	ApplicationRuntime::start/ runUntilStopped/stop	系统启动信号、停止信号	统一启动 UI、解析、媒体、IPC 和模拟采集线程
协议解析	ElevatorFrameParser::parseByte	单字节串口数据	校验通过后输出 ElevatorEnvData
单片机数据源	ElevatorEnvSimulator::runLoop	模拟楼层/温湿度序列	与单片机一致的协议帧；实物接入时可替换为串口接收
本地 UI	ElevatorUiController::postData/ applyPendingData	环境数据、报警命令	刷新楼层、温湿度、红色警报提示
本地上报	UdpTelemetryClient::publishEnvData/ publishMediaStatus	环境数据、媒体事件	发送 JSON 到 /tmp/elevator_telemetry.sock
本地命令	UdpCommandServer::listenLoop	ALARM/PLAY_AD/START_CAMERA 等命令	调用 UI、广告、摄像头控制回调

媒体管理	MediaManager::startCamera/playAdLoop/stopAd	云端命令、媒体状态	协调摄像头检测和广告播放
摄像头推流	CameraStreamService::start	摄像头帧、RKNN 检测结果	叠加目标框并推送 RTSP
AI 推理	RknnDetector::infer	OpenCV BGR 帧	检测框、类别和置信度
广告播放	VideoAdPlayer::playLoop/stop	本地 MP4 文件	KMS 区域循环播放或清屏
云端桥接	mqtt_client.py	Socket 数据、MQTT 指令	发布 telemetry/media/state, 转发 command
网页平台	云端.html	MQTT WebSocket、上传接口	监控实时数据、下发控制命令

第三部分 完成情况及性能参数

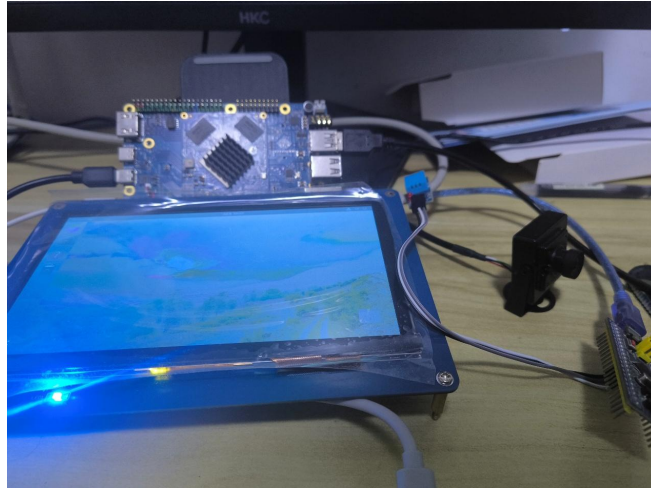
3.1 整体介绍

作品已完成边缘端主程序、单片机数据协议模拟、本地 LVGL 显示、Unix Socket 本地通信、MQTT 云端桥接、网页端监控与控制、广告上传投放、摄像头推流和 RKNN 检测集成。当前工程既能在 PC/仿真环境进行界面与协议验证，也能通过 ARM64 交叉编译部署到开发板运行。

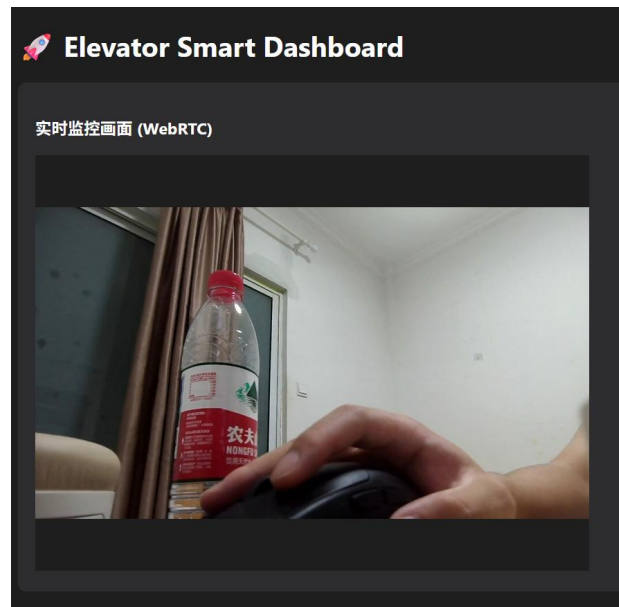
3.2 工程成果（分硬件实物、软件界面等设计结果）

系统整体效果图：

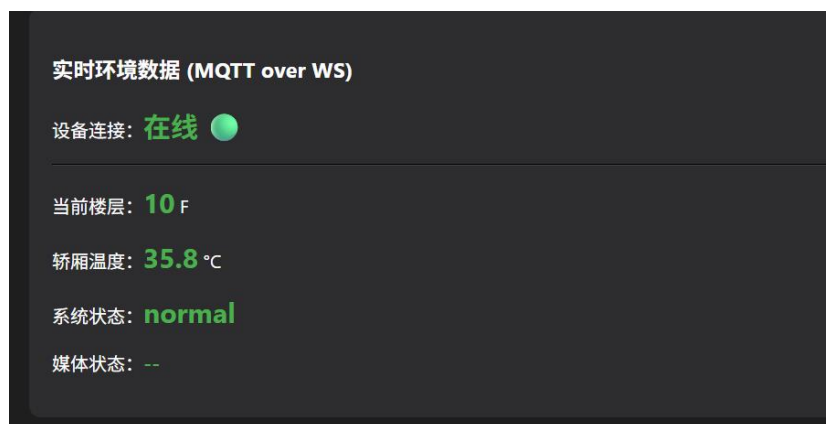




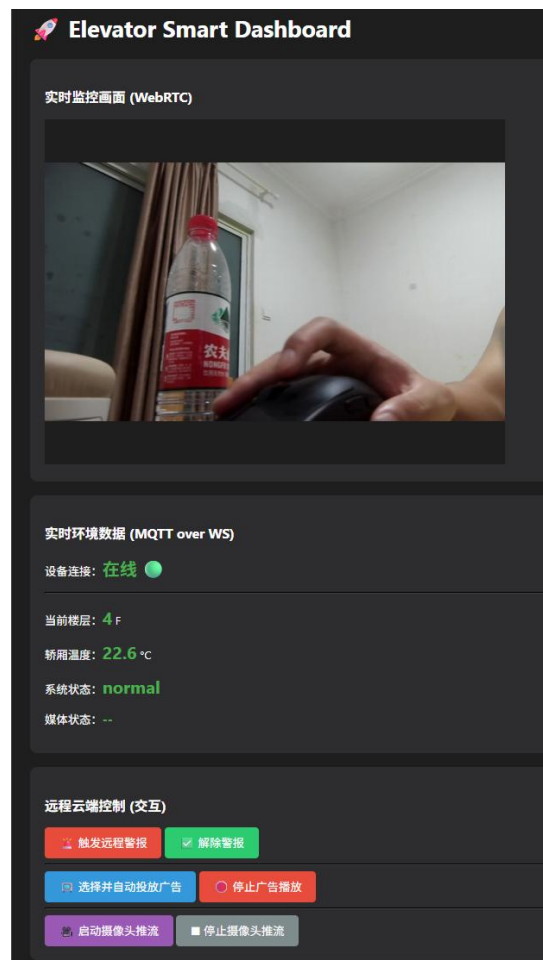
软件界面：



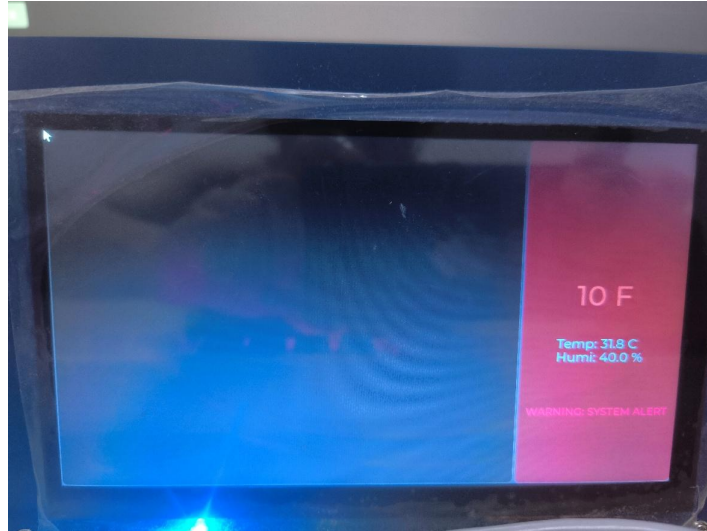
实时数据：



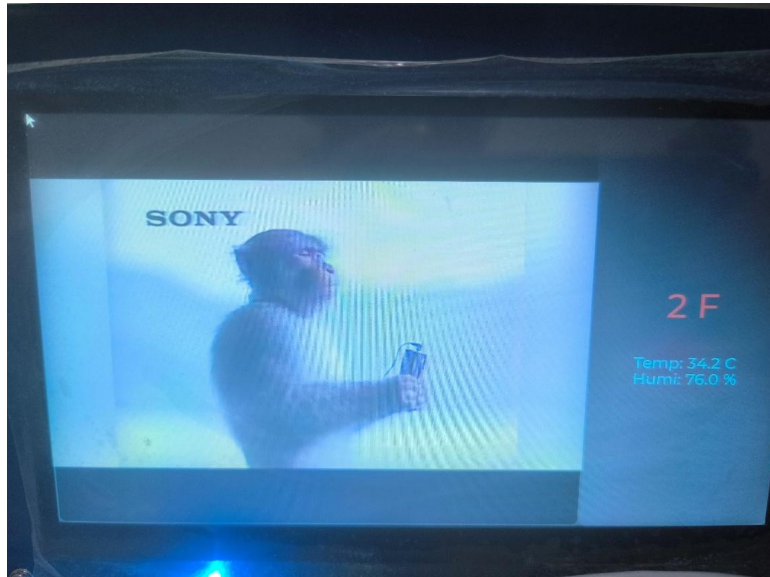
远程交互：



功能展示:
触发报警:



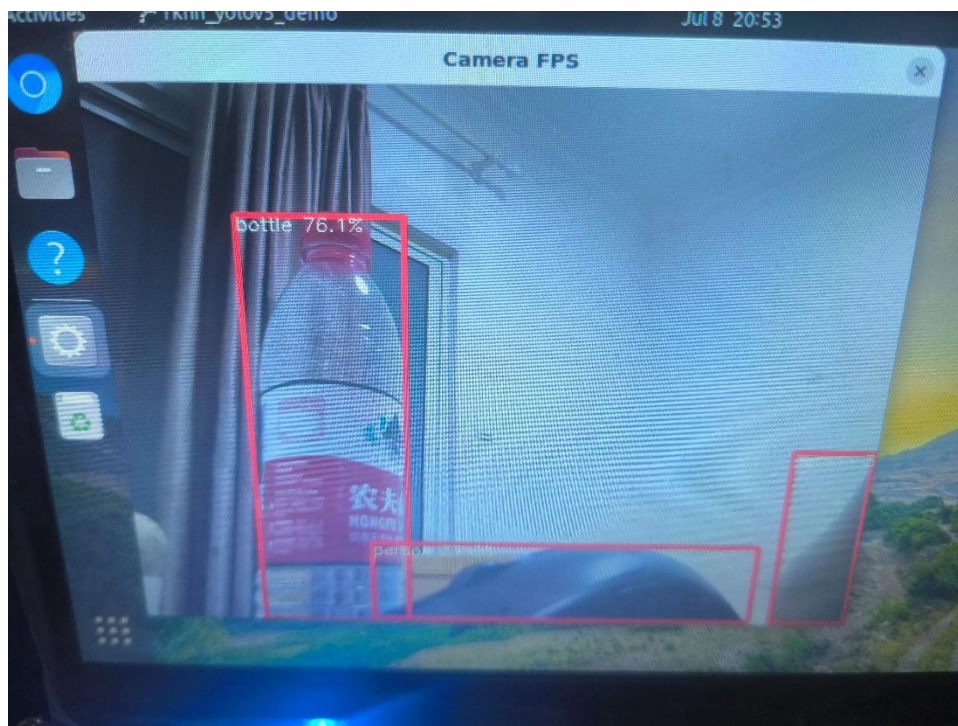
云端投放广告视频:



摄像头 yolov5 模型目标检测:

```

elf@elf2-desktop:~/yolov5_video/rknn_yolov5_demo_Linux$ ./rknn_yolov5_demo model/RK3588/yolov5s.rknn
模型名称:      model/RK3588/yolov5s.rknn
线程数: 6
Loading mode...
model is NHWC input fmt
Loading mode...
rga_api version 1.8.0_[0]
loadLabelName ./model/coco_80_labels_list.txt
model is NHWC input fmt
Loading mode...
model is NHWC input fmt
Loading mode...
model is NHWC input fmt
Loading mode...
model is NHWC input fmt
Loading mode...
model is NHWC input fmt
60帧平均帧率: 17.187052帧
60帧平均帧率: 16.268980帧
60帧平均帧率: 16.264570帧
60帧平均帧率: 16.268980帧
60帧平均帧率: 16.251354帧
60帧平均帧率: 16.264570帧
60帧平均帧率: 16.277808帧
60帧平均帧率: 16.264570帧
60帧平均帧率: 16.273393帧
    
```

3.3 特性成果

测试项	测试方法	结果
协议解析	输入含 CRC16 的模拟单片机帧	能正确解析楼层、温度、湿度、帧序号和时间戳
错误帧处理	构造 CRC 错误或帧尾缺失数据	解析器丢弃异常帧并回到等待帧头状态
本地显示	周期性输入采集数据	LVGL 界面能刷新楼层和环境信息，报警时显示红色警告
云端上报	运行 mqtt_client.py 监听 telemetry Socket	网页端能看到楼层、温度、状态和在线信息
远程报警	网页点击触发/解除警报	C++ 收到 ALARM_TRIGGER/ALARM_CANCEL 并更新 UI
广告投放	网页上传 MP4 并下发 switch_ad	边缘端下载文件并循环播放，停止后清屏
摄像头推流	网页下发 start_camera	边缘端启动 1080p30 采集、编码并推到 RTSP 地址
AI 叠加	启用 RKNN 模型	检测结果以目标框和置信度叠加在视频画面中

第四部分 总结

4.1 可扩展之处

后续可从五个方向扩展：一是接入真实电梯控制器或传感器总线，如 RS485、CAN 或继电器状态量，实现更完整的门状态、载重、故障码和运行方向采集；二是扩展 AI 模型类别，加入人员滞留、跌倒、烟雾、危险动作和拥挤度识别；三是增加数据库与历史趋势分析，实现温湿度、故障和媒体状态的长期留存；四是加入权限认证、TLS、设备证书和命令签名，提高云端控制安全性；五是完善 OTA 更新和远程日志，便于批量部署后的维护。

4.2 心得体会

本作品的开发过程体现了嵌入式系统从底层采集到云端应用的完整工程链路。最初设计时，单片机采集、图形界面、视频推流和云端控制看似是相互独立的功能，但真正联调时会遇到线程同步、数据格式、媒体资源占用和网络状态变化等问题。因此系统采用了分层解耦方式：解析器只负责把字节流变成结构化数据，UI 通过线程安全队列消费最新状态，媒体模块只暴露启动、停止和播放接口，云端桥接脚本负责协议转换。这样每个模块都能独立调试，出现问题时也更容易定位。

在通信设计上，作品没有直接传输松散字符串，而是设计了带 CRC16 校验的二进制帧，并加入帧序号和设备时间戳。这一做法让模拟数据与真实单片机数据保持一致，既方便早期无硬件时开发，也方便后期接入真实传感器。在媒体设计上，摄像头、AI 推理、RTSP 推流和广告播放都涉及较高资源占用，系统通过 MediaManager 做统一调度，广告播放时关闭检测，停止广告后恢复检测，避免多个功能争夺显示和计算资源。

通过本次作品实现，可以明显感受到嵌入式竞赛项目不只是把某个功能做出来，更重要的是把数据流、控制流、异常处理和展示效果连成一个可靠系统。后续若继续完善实物结构、电路板和云端数据分析，本作品可以进一步发展为面向。

第五部分 参考文献

- [1] GB/T 7588.1-2020, 电梯制造与安装安全规范 第 1 部分：乘客电梯和载货电梯[S].
- [2] OASIS. MQTT Version 5.0 Specification[S]. 2019.
- [3] LVGL Documentation. LVGL embedded graphics library[EB/OL].
- [4] FreeRTOS Kernel Documentation. Real-time operating system kernel[EB/OL].
- [5] GStreamer Application Development Manual. Multimedia framework

documentation[EB/OL].

[6] OpenCV Documentation. Open source computer vision library[EB/OL].

[7] Rockchip RKNN Toolkit2 User Guide. RKNN model conversion and deployment guide[Z].

[8] EMQX Documentation. MQTT broker and MQTT over WebSocket guide[EB/OL].

[9] Flask Documentation. Web application framework for Python[EB/OL].

[10] CMake Documentation. Cross-platform build system manual[EB/OL].

[11] ISO/IEC 9899:2018. Programming languages C[S].

[12] ISO/IEC 14882:2020. Programming languages C++[S].